

1 39. Building Polygonal Models of Surfaces

To get started, following points make it a bit easier:

- Keep **polygon** orientations consistent. Make sure that when viewed from the outside, all the **polygons** on the surface are oriented in the same direction (all **clockwise** or all **counterclockwise**). Try to get this right the first time, since it's excruciatingly painful to fix the problem later. (If you use `glScale*`() to reflect geometry around some axis of symmetry, you might change the orientation with `glFrontFace()` to keep the orientations consistent.)
- When you subdivide a surface, watch out for any nontriangular polygons. The three **vertices** of a **triangle** are guaranteed to lie on a plane; any polygon with four or more **vertices** might not. Nonplanar **polygons** can be viewed from some orientation such that the edges cross each other, and OpenGL might not render such polygons correctly.
- There's always a trade off between performance and quality. Use small **polygons** for curvatures and less **polygons** for flat surfaces.
- For high-quality images, it's a good idea to subdivide more on the outer edges than in the interior.
- Try to avoid T-intersections in your models.
- If you're constructing a closed surface, make sure to use exactly the same numbers for **coordinates** at the beginning and end of a closed loop, or you can get gaps and cracks due to numerical round-off.

Example of *bad* 2D code:

```
#define PI 3.14159265
#define EDGES 30

/*draw a circle*/
glBegin(GL_LINE_STRIP);
for (i = 0; i <= EDGES; i++)
    glVertex2f(
        cos((2 * PI * i) / EDGES),
        sin((2 * PI * i) / EDGES)
    );
glEnd();
```

Fixes:

- When `i == EDGES`, use 0 for `cos()` and `sin()` functions instead of `(2 * PI * EDGES / EDGES)`.
- Use `GL_LINE_LOOP` instead of `GL_LINE_STRIP` and change the loop termination condition to `i < EDGES`.

1.1 An Example

1.1.1 Building an Icosahedron

```
#define X 0.525731112119133606
#define Z 0.850650808352039932

static GLfloat vdata[12][3] = {
    {-X, 0.0, Z}, {X, 0.0, Z}, {-X, 0.0, -Z}, {X, 0.0, -Z},
    {0.0, Z, X}, {0.0, Z, -X}, {0.0, -Z, X}, {0.0, -Z, -X},
    {Z, X, 0.0}, {-Z, X, 0.0}, {Z, -X, 0.0}, {-Z, -X, 0.0}
};

static GLuint tindices[20][3] = {
    {0, 4, 1}, {0, 9, 4}, {9, 5, 4}, {4, 5, 8}, {4, 8, 1},
    {8, 10, 1}, {8, 3, 10}, {5, 3, 8}, {5, 2, 3}, {2, 7, 3},
```

```

    {7, 10, 3}, {7, 6, 10}, {7, 11, 6}, {11, 0, 6}, {0, 1, 6},
    {6, 1, 10}, {9, 0, 11}, {9, 11, 2}, {9, 2, 5}, {7, 2, 11}
};

int i;
glBegin(GL_TRIANGLES);
for (i = 0; i < 20; i++) {
    // color information here
    glVertex3fv(&vdata[tindices[i][0]][0]);
    glVertex3fv(&vdata[tindices[i][1]][0]);
    glVertex3fv(&vdata[tindices[i][2]][0]);
}
glEnd();

```

X and Z are chosen such that the distance from the origin to any of the vertices of the icosahedron is `1.0`. The coordinates of vertices are given in `vdata[] []` where the zeroth vertex is $(-X, 0.0, Z)$. The first triangle is made from vertices

1. Zeroth
2. Fourth
3. First

1.1.2 Calculating Normal Vectors for a Surface

The normal vector¹ for all 3 vertices of the triangle are same.

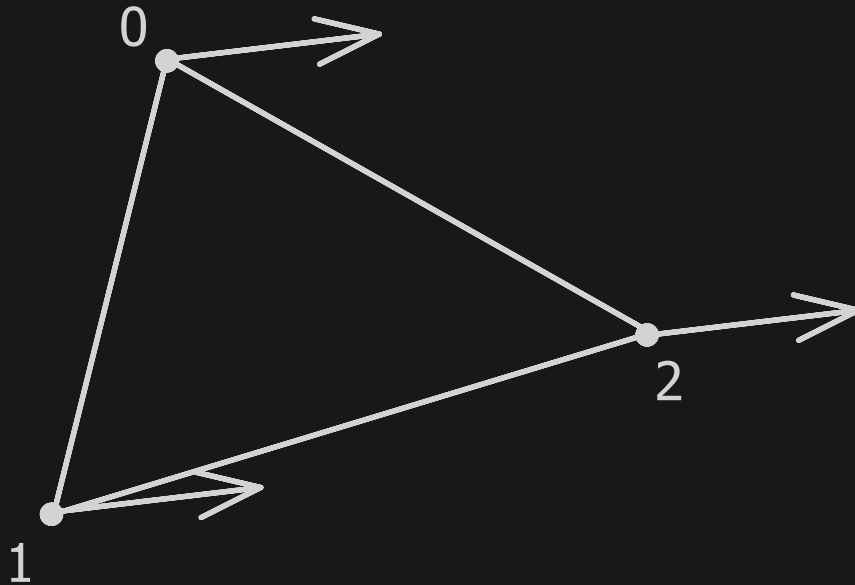


Figure 1: A triangle with vectors normal to all of its vertices.

```

void normalize(float v[3]) {
    GLfloat d = sqrt(v[0] * v[0] + v[1] * v[1] + v[2] * v[2]);

```

¹Read more about [surface normals](#).

```

    if (d == 0.0) {
        error("zero length vector");
        return;
    }
    v[0] /= d;
    v[1] /= d;
    v[2] /= d;
}

void normcrossprod(float v1[3], float v2[3], float out[3]) {
    out[0] = v1[1] * v2[2] - v1[2] * v2[1];
    out[1] = v1[2] * v2[0] - v1[0] * v2[2];
    out[2] = v1[0] * v2[1] - v1[1] * v2[0];
    normalize(out);
}

GLfloat d1[3], d2[3], norm[3];

for (int j = 0; j < 3; j++) {
    d1[j] = vdata[tindices[i][0]][j] - vdata[tindices[i][1]][j];
    d2[j] = vdata[tindices[i][1]][j] - vdata[tindices[i][2]][j];
}

normcrossprod(d1, d2, norm);
glNormal3fv(norm);

```

Here is the code that would draw an icosahedral approximation of a smoothly shaded sphere

```

glBegin(GL_TRIANGLES);
for (int i = 0; i < 20; i++) {
    glNormal3fv(&vdata[tindices[i][0]][0]);
    glVertex3fv(&vdata[tindices[i][0]][0]);

    glNormal3fv(&vdata[tindices[i][1]][0]);
    glVertex3fv(&vdata[tindices[i][1]][0]);

    glNormal3fv(&vdata[tindices[i][2]][0]);
    glVertex3fv(&vdata[tindices[i][2]][0]);
}
glEnd();

```

1.1.3 Improving the Model



Figure 2: Subdividing to improve approximation

Source: perso.esiee.fr/~perrotol/TheRedBook-1.0/figures/icos.gif

Single subdivision code

```
void drawtriangle(float* v1, float* v2, float* v3) {
    glBegin(GL_TRIANGLES);
    glNormal3fv(v1); glVertex3fv(v1);
    glNormal3fv(v2); glVertex3fv(v2);
    glNormal3fv(v3); glVertex3fv(v3);
    glEnd();
}

void subdivide(float* v1, float* v2, float* v3) {
    GLfloat v12[3], v23[3], v31[3];
    for (int i = 0; i < 3; i++) {
        v12[i] = v1[i] + v2[i];
        v23[i] = v2[i] + v3[i];
        v31[i] = v3[i] + v1[i];
    }
    normalize(v12);
    normalize(v23);
    normalize(v31);

    drawtriangle(v1, v12, v31);
    drawtriangle(v2, v23, v12);
    drawtriangle(v3, v31, v23);
    drawtriangle(v12, v23, v31);
}

for (int i = 0; i < 20; i++) {
    subdivide(
        &vdata[tindices[i][0]][0],
        &vdata[tindices[i][1]][0],
        &vdata[tindices[i][2]][0]
    );
}
```

recursive subdivision code

```
void subdivide(float* v1, float* v2, float* v3, long depth) {
    GLfloat v12[3], v23[3], v31[3];

    if (depth == 0) {
        drawtriangle(v1, v2, v3);
        return;
    }

    for (int i = 0; i < 3; i++) {
        v12[i] = v1[i] + v2[i];
        v23[i] = v2[i] + v3[i];
        v31[i] = v3[i] + v1[i];
    }

    normalize(v12);
    normalize(v23);
    normalize(v31);

    subdivide(v1, v12, v31, depth - 1);
    subdivide(v2, v23, v12, depth - 1);
    subdivide(v3, v31, v23, depth - 1);
    subdivide(v12, v23, v31, depth - 1);
}
```

1.2 Generalized Subdivision

Typically, the recursion ends either if

- a certain `depth` is reached
- if some condition on the `curvature` is satisfied

To look at a more generalized solution to subdivision problem, consider an arbitrary surface parameterized using two variables `u[0]` and `u[1]`.

The following function takes `u[]` and returns 3D vertex and normal unit vectors²

```
void surf(GLfloat u[2], GLfloat vertex[3], GLfloat normal[3]);
```

The following function takes `u[]` and returns the curvature of the surface at that point.

```
float curv(GLfloat u[2]);
```

Following code shows the recursive routine that subdivides a triangle either until the maximum `depth` is reached or until the maximum curvature at the three vertices is less than some cutoff.

```
void subdivide(float u1[2], float u2[2], float u3[2], float cutoff, long depth) {
    GLfloat v1[3], v2[3], v3[3];
    GLfloat n1[3], n2[3], n3[3];
    GLfloat u12[2], u23[2], u31[2];

    if (depth == maxdepth || (curv(u1) < cutoff && curv(u2) < cutoff && curv(u3) < cutoff)) {
```

²Read more about [surface normals](#).

```
    surf(u1, v1, n1);
    surf(u2, v2, n2);
    surf(u3, v3, n3);

    glBegin(GL_POLYGON);
    glNormal3fv(n1); glVertex3fv(v1);
    glNormal3fv(n2); glVertex3fv(v2);
    glNormal3fv(n3); glVertex3fv(v3);
    glEnd();

    return;
}

for (int i = 0; i < 2; i++) {
    u12[i] = (u1[i] + u2[i]) / 2.0f;
    u23[i] = (u2[i] + u3[i]) / 2.0f;
    u31[i] = (u3[i] + u1[i]) / 2.0f;
}

subdivide(u1, u12, u31, cutoff, depth + 1);
subdivide(u2, u23, u12, cutoff, depth + 1);
subdivide(u3, u31, u23, cutoff, depth + 1);
subdivide(u12, u23, u31, cutoff, depth + 1);
}
```